Review changelog, batch p6 (Phase 6: Data fetching and state ecosystems, lessons 01-20)

Date: 2026-10-22. Reviewer scratch work: `/Users/todorbra/Projects/ReactLearning/tools/sandbox/scratch/p6/` (vitest/jsdom tests, node scripts, patch scripts under `patches/`).

Versions exercised: React / react-dom 19.3.1, `@tanstack/react-query` and `@tanstack/query-core` 5.104.5 (plus `@tanstack/react-query-persist-client`, `query-persist-client-core`, `query-async-storage-persister` 5.104.5, read from typings), `@reduxjs/toolkit` 2.13.0, `react-redux` 9.3.0, `zustand` 5.0.16, `xstate` 5.33.3 (`@xstate/react` 6.1.0 read from source), `zod` 4.6, `react-error-boundary` 6.1.6 (typings + executed), `react-router` 8.5.0 (typings and changelog), `serialize-javascript` 7.1.2, ESLint 10.13, Node 24.16. Browser support figures come from `@mdn/browser-compat-data`. react.dev pages fetched and read: useOptimistic, useTransition, use, Suspense, cache, createRoot, useSyncExternalStore, form, renderToString, React 19 blog and upgrade guide, React 19.2 blog.

Method: all 240 TS/TSX code blocks in the lessons were extracted and typechecked (`tsc --noEmit`, strict, with `any` stubs for missing app-local names); no library-API type errors in the TanStack/RTK/Zustand/XState snippets (remaining errors were undeclared app code, plus intentionally non-compilable BAD-example fragments). All 131 external URLs were fetched: every one returned HTTP 200 (the doc sites return 404 for bad paths, so the 200s are meaningful); the former "(unverified deep link)" tags were replaced with "link checked, HTTP 200, 2026-10-22". Zero warning markers remain in the lessons (every one was either verified and removed, or turned into an explicit "not verified in this review" hedge).

Result totals (claims individually checked): about 115 confirmed, 24 corrected, 16 unresolved (listed at the end).

---

01 Fetch lifecycle

- Confirmed (executed): TanStack `status` / `fetchStatus` names and idle/pending values for a fresh or disabled observer; `Response.json()` typed any; StrictMode runs an extra effect setup/cleanup (test: 2 requests and 1 `setState` with an `ignore` flag); `use` needs a cached promise (react.dev `use` reference quotes).
- Corrected: `useEffectEvent` described as volatile; it is exported by stable `react` 19.2+ (19.3 here) and documented on react.dev (with call-site restrictions). Added the React 19 StrictMode `useMemo` / `useCallback` reuse note (upgrade guide).
- Hedged: framework request-scoped memoisation and "several cache layers" now name `cache` in Server Components and point to framework docs.

02 Cancellation and race conditions

- Confirmed (executed in Node 24 against a local server): default `abort()` rejects with `AbortError` identical to `signal.reason`; custom reason is passed through; `AbortSignal.timeout` gives `TimeoutError`; `AbortSignal.any`; `throwIfAborted`; body read after abort rejects `AbortError`. Race table (R 400 ms, RE 600 ms, REA 100 ms): naive hook ends on "re", ignore flag and abort end on "rea" (RTL with fake timers). Serial queue code keeps order and survives a rejection.
- Corrected: section 5.8 claimed the query library "discards the result for an unobserved or cancelled query". Executed against query-core 5.104: that is true for `cancelQueries` (entry reverts, late result discarded), but when the last observer unsubscribes and `queryFn` ignores `signal`, the request runs to completion and the result IS written to the cache. With a consumed signal, unsubscribing aborts. Text rewritten with the three observed cases.
- Added MDN support versions for `AbortSignal.timeout` / `any` / `throwIfAborted`; cited the useTransition caveat that updates after `await` need an extra `startTransition`.
- Unresolved: whether specific routers cancel loaders on navigation (left as "check your router version").

03 Loading and error states

- Confirmed: error boundaries still class-only; `createRoot` `onCaughtError(error, errorInfo)` executed (componentStack present), `onUncaughtError` / `onRecoverableError` confirmed in the react.dev reference; an error thrown inside `startTransition(async () => ...)` reaches the nearest boundary; event-handler throws do not; `useFormStatus` placement rule; TanStack v5 `isLoading = isFetching && isPending`, `isInitialLoading` deprecated; `react-error-boundary` v6 API (`fallbackRender`, `onReset`, `resetKeys`, `useErrorBoundary`).
- Corrected: lesson said fallback-reveal throttle durations are internal and must not be asserted. The react.dev Suspense reference now documents "at most once every 300 ms" (stated with a warning not to build logic on it).

04 API normalization

- Confirmed (executed): date-only ISO parses as UTC midnight and prints as the previous day in `America/Los_Angeles`; date-time without offset is local; Zod snippet runs, strips unknown keys, reports `issues[].path`; `replaceEqualDeep` keeps identity; RTK has `createEntityAdapter`, Reselect `createSelector`.
- Corrected: snippet updated to Zod 4 (`z.iso.datetime`, `.int()`; `.datetime()` and `.safe()` are deprecated in 4.x typings). Added a note that V8's `Date.parse` does not return NaN for impossible days such as `02-31` (it rolls over); the `isoInstant` parser comment now says so.
- Added Temporal support figures (Chrome 144, Firefox 139, Safari preview; absent in Node 24).
- Unresolved: SWR default comparison behaviour, Apollo/urql/Relay normalisation claims (hedged as not verified).

05 Optimistic updates

- Confirmed (React 19.3, RTL): `addOptimistic` outside an Action logs a `console.error` (and the value is not shown); several `addOptimistic` calls in one Action apply in order; overlay drops on success and on throw; `<form action>` function: uncontrolled fields reset after success, kept when

the action throws; no warning when `addOptimistic` is called from a form action; TanStack `cancelQueries`, `useMutationState`, `variables` / `isPending` exist.
- Corrected: section 3.2 predicted a "flash of the pre-mutation UI" when the post-`await` update is not in a transition. Observed render sequence is actually a transient DUPLICATE (confirmed row and overlay row both visible, `hi, hi*`, and duplicate keys if they share `clientKey`) before the overlay drops; with `startTransition` it is clean (`hi*` then `hi`). Text rewritten; the old description kept as the opposite-timing glitch.
- Corrected/added: the "invalidate only when the last mutation settles" advice now states the verified check `isMutating({ mutationKey }) === 1` inside `onSettled` (the settling mutation still counts: observed 2 then 1). Documented the 5.104 callback signatures (`onMutate(variables, context)`, `onError(error, variables, onMutateResult, context)`).
- Added `crypto.randomUUID` support (Chrome 92, Firefox 95, Safari 15.4).

06 Cache consistency and invalidation

- Confirmed (query-core 5.104): key hashing ignores object-key order but not array order; prefix and partial-object matching; `refetchType` values and default `'active'`; inactive entries only marked invalid and refetched on remount; `clear()` empties the cache; `retry` / `gcTime` defaults; RTK Query `keepUnusedDataFor` default 60 s and `invalidationBehavior` default `'delayed'`; `BroadcastChannel`.
- Corrected: `invalidateQueries` "cancels and refetches in-flight fetches by default" is only true when the entry already has data (background refetch: second fetch started and first signal aborted; `cancelRefetch: false` kept the first). If the INITIAL fetch is in flight (no data yet), invalidation joins it, so a pre-mutation response can land. Added.
- Corrected: `BroadcastChannel` does not deliver to the posting channel object, but other objects with the same name in the same page do receive (executed).
- Unresolved: Next.js/Apollo/SWR cache-table cells (hedged), `stale-if-error` support across CDNs.

07 Server-state management

- Confirmed (executed): `initialData` is cached and respects `staleTime`; `placeholderData` is never cached; hydrated entries refetch with `staleTime: 0` and not with 60 000; `select` output is structurally shared; `useSuspenseQuery` in one component serialises its reads while sibling components start together (React 19 pre-warming); mutation `retry` default 0; `enabled`, `queryOptions`, `useMutationState`, `getQueriesData`, `isPlaceholderData`, `keepPreviousData` all exist; tracked properties.
- Corrected: answer A5 said a placeholder query "is still pending". In v5 the observer reports `status: 'success'`, `isPlaceholderData: true`, `fetchStatus: 'fetching'`. Fixed in §2.7 and A5.
- Added MDN: `NetworkInformation.saveData` Chrome 65 only. Quoted the react.dev `useSyncExternalStore` caveat that store updates cannot be transitions.
- Unresolved: the responsibility table cells for RTK Query, Apollo/urql, framework columns (explicitly marked not re-verified).

08 TanStack Query

- Confirmed: `staleTime` default 0 (and `'static'` exists), `gcTime` 5 min client / Infinity server, retry 3 with `min(1000*2^n, 30000)` (scaled-delay run: 4 attempts at 1:2:4 spacing), server default 0 retries, no `onSuccess` on queries, `initialPageParam` required, `maxPages`, `skipToken`, default `TError = Error`, `useBaseQuery` uses `useSyncExternalStore`, tracked props (component reading only `data` did not re-render on `isFetching`), `undefined` data becomes an error, cache subscribe event shape, StrictMode does not double-fetch, returning the `invalidateQueries` promise keeps the mutation `pending` (verified both ways), `queryOptions` tags the key.
- Corrected/updated: banner now states checked 5.104 and notes the newer mutation callback signature (`onMutateResult`, trailing `context`).

09 Redux Toolkit

- Confirmed (executed): slice reducers share structure (untouched `items[0]` identical); mutate-and-return-new throws; mutate-and-return-same-draft is fine; slice `selectors` work with `combineSlices`; `createAsyncThunk` abort gives `meta.aborted` / `AbortError` and `condition:false` gives `meta.condition` / `ConditionError`; RTK Query tag invalidation removes an unsubscribed entry without refetching; optimistic `updateQueryData` plus `undo()`; React-Redux 9 `stabilityCheck` / `identityFunctionCheck` default `"once"`; Reselect 5 `weakMapMemoize` default.
- Corrected: "Immer auto-freeze is development-only" is wrong. State was frozen with `NODE_ENV=production` too; the bad example comment ("in prod: silent corruption") now says it throws a TypeError in strict-mode code in both environments.
- Corrected: default middleware is 4 entries in dev (thunk, immutable, serializable, action-creator check) and 1 in production (verified).
- Added: RTK Query invalidates tags even when the mutation FAILS (refetches observed after a failed PATCH); return `[]` from `invalidatesTags` on error to avoid it.
- Unresolved: whether `resetApiState()` aborts the in-flight network request (cache stayed empty; abort itself not observed).

10 Zustand

- Confirmed (executed): v5 selector returning a new object throws "Maximum update depth exceeded" after "The result of getSnapshot should be cached"; `useShallow` avoids re-render for unrelated keys; `createWithEqualityFn` exists in `zustand/traditional`; `setState(s => s)` is a no-op, a repeated fresh partial still notifies; `setState(partial, true)` requires the full state in typings; single-call `create<S>(fn)` typechecks; `persist` with sync storage hydrates synchronously during `create`, `migrate(persisted, version)` and `partialize` work, `skipHydration` / `hasHydrated` exist; `subscribeWithSelector`.
- Corrected: claim that v5 reduced the `use-sync-external-store` dependency made precise (v5 `create` uses `React.useSyncExternalStore`; the shim is an optional peer for `zustand/traditional`).
- Unresolved: middleware ordering recommendations beyond `devtools` outermost (docs-based, not executed).

11 Context vs external stores

- Confirmed (executed): children element bail-out (0 child renders after provider state change, 1 when written inline); memoised consumers still re-render on Context change regardless of which field they read; a store update inside `startTransition` is applied synchronously (store value new while React state still old and `isPending` true); React 19 `<Ctx value>` syntax, `.Provider` slated for deprecation (19 blog).
- Unresolved: React Compiler interaction with Context, `use-context-selector` status (hedged).

12 State machines and XState

- Confirmed (xstate 5.33 executed): toggle emits `off,on,off`; `setup().createMachine`, `fromPromise({ input, signal })` aborted when the state exits (CANCEL: aborted true, machine stays idle, late result not delivered); retry guard/`exhausted` flow; `final` gives `status: 'done'`; exit, transition, entry order; actions run in written order so `assign` is visible to later actions; `reenter: true` debounce and abort-on-INPUT; `.provide`; `waitFor`; `hasTag`; `getPersistedSnapshot` restore; `stateIn` exported; `@xstate/react` default comparator `===` and Strict Mode handled by stop/restart with persisted snapshot.
- Corrected: banner said "no `interpret`"; `interpret` still exists in 5.33 as a deprecated alias. `actor.send('TOGGLE')` throws a TypeError. Added: subscribers are notified even for an event no transition handles (same snapshot object), so subscribe is not a change signal.

13 Suspense data loading

- Corrected (significant): the lesson said React stops at the first suspended sibling so sibling `useSuspenseQuery` calls form a Suspense-caused waterfall. In React 19 the fallback commits and suspended siblings are then rendered to pre-warm (upgrade guide), verified on 19.3: two sibling components using `use(cachedPromise)` started 1-2 ms apart; a parent that suspends before rendering its child still waterfalls (child request started ~150 ms later); two reads in one component still serialise. Section rewritten.
- Corrected: fallback throttle (300 ms documented); `SuspenseList` is not exported by stable `react`; `useSuspenseQuery` also omits `throwOnError`, `placeholderData` and `skipToken` (typings), not only `enabled`; an uncached `use(new Promise())` never resolves (18 renders, fallback still showing after ~320 ms).
- Confirmed: `cache` is Server-Components-only per react.dev.

14 Server-side data loading

- Confirmed (executed): default `dehydrate` omits failed and pending queries (`shouldDehydrateQuery` including pending shows streaming possibility); `prefetchQuery` resolves undefined on error, `fetchQuery` rejects; `isServer`, `dehydrate`, `HydrationBoundary` exported; `Date` stays a Date in `dehydrate` and becomes a string after JSON; `serializeData` / `deserializeData` options exist; `serialize-javascript` `isJSON` escapes `</script>` and U+2028; react.dev: `renderToString` "does not support streaming or waiting for data"; `cache` Server-Components-only; taint APIs are experimental-only.

- Corrected: `renderToString` was called "legacy"; reworded to match the react.dev pitfall.
- Unresolved: Next.js/React Router/TanStack Start specifics (not executed).

15 Error recovery and retries

- Confirmed (executed): retry timeline (4 attempts, 1/2/4 s spacing, `isError` at ~7.4 s); retry callback receives 0 for the first failure; mutation retry default 0; function-valued `refetchInterval`; `failureCount` / `failureReason`; `QueryErrorResetBoundary` + `react-error-boundary` v6 `resetKeys` / `onReset` pattern (first load fails, "Try again" refetches, `resetKeys` recovers); without `onReset` the boundary re-throws immediately with no request (answer B4 confirmed).
- Added: `FallbackProps.error` is `unknown` in v6; root callbacks confirmed; Web Locks support versions.

16 Offline considerations

- Confirmed (executed, onlineManager.setOnline(false)): default `networkMode: 'online'` leaves a query `pending` / `paused` with zero `queryFn` calls and a mutation `isPaused`; `'always'` fetches; `'offlineFirst'` makes one attempt then pauses retries; persistence option names (`persister`, `maxAge`, `buster`, `dehydrateOptions.shouldDehydrateQuery`, `createAsyncStoragePersister({ storage, throttleTime })`) read from 5.104 typings; `setMutationDefaults` / `resumePausedMutations` exist.
- Corrected: the lesson said the library "trusts `navigator.onLine`'s `false`". In 5.104 `onlineManager` starts as `true` and flips only on window `online` / `offline` events (it does not read `navigator.onLine`).
- Added MDN support: OPFS (`FileSystemDirectoryHandle`), `navigator.locks`, `saveData`.
- Unresolved: storage quota and WebKit eviction specifics, `idb` / `idb-keyval`, `Clear-Site-Data` browser support.

17 Real-time

- Corrected (code bug, found by running the class with a fake WebSocket and fake timers): `RealtimeClient` re-armed the pong watch on every message, including the pong itself, so it closed a healthy connection 10 s after every pong. Fixed: the watch is armed only when a ping is sent and cleared (`clearPongWatch`) on any incoming message. Verified: answering server stays open for 100 s; silent server is closed and status becomes `reconnecting`.
- Confirmed/added: `WebSocketStream` Chrome-only, `WebTransport` support versions (MDN), `useEffectEvent` stable in 19.2+, `refetchIntervalInBackground` exists.
- Unresolved: browser behaviour of `close()` on a half-open socket (hedged), WebSocket/SSE protocol facts not run against a live server.

18 URL state management

- Confirmed (jsdom, React 19.3): hand-rolled `useSearchParam` notifies both components, `pushState` / `replaceState` fire no `popstate`; the debounced SearchBox effects do not fight

(external change to `?q=x` leaves input `x` with no stray commit); `URLSearchParams` behaviour in P5.
- Corrected: impossible dates. Lesson (and its model answer) said `2025-02-31T00:00:00Z` parses as NaN; V8 rolls it over to 1 March. The model-answer code now adds a round-trip check; verified it rejects `2025-02-31` and `2025-04-31`.
- Corrected: React Router import note. `react-router` v8.5 is current; `react-router-dom` is removed in v8 (compat re-export in v7). Confirmed from the 8.5 typings that the functional `setSearchParams` form does NOT queue like `setState`, and `searchParams` is a stable reference.
- Added: Navigation API now ships in all three engines (Chrome 102, Firefox 147, Safari 26.2 per MDN data).
- Unresolved: TanStack Router example, `nuqs`, Next.js `useSearchParams` behaviour.

19 Technology selection matrix

- Confirmed: ESLint flat-config `no-restricted-imports` sketch executed on ESLint 10.13 (flags `useQuery` outside ignored folders); TanStack/Zustand/RTK/XState snippets typecheck.
- Hedged: bundle size, maintenance status, SWR/atoms libraries, framework data layers (explicitly marked not verified).

20 Phase assessment

- Confirmed (executed): P1 (2 requests, 1 `setUser`), P2 (gcTime vs staleTime timeline: GC'd entry means pending + request; surviving entry means fresh, no request), P3 (3 attempts, `isError` at about 3.3 s, `pending` / `fetching`, `failureCount` 2, 404 stops after one), P4 (Context: both consumers render; Zustand: only `B2`), P5 values, Q8 (v5 selector loop), `isMutating === 1` semantics, `AbortSignal.any` support.
- Updated the version/verification banner accordingly.

Unresolved (could not verify here)

1. SWR, Apollo, urql, Relay behaviours and defaults (cache tables in 04, 06, 07, 19).
2. Next.js/React Router/TanStack Start data-layer and revalidation APIs (06, 07, 14, 18, 19).
3. Browser storage quotas and WebKit eviction policy; `Clear-Site-Data` support (16, 05).
4. Browser behaviour of `ws.close()` on half-open sockets; WebSocket/SSE close-code and `Last-Event-ID` facts not run against a live server (17).
5. React Compiler effect on Context re-renders; `use-context-selector` status (11).
6. Whether RTK Query `resetApiState()` aborts in-flight HTTP requests (09).
7. TanStack Router `validateSearch` example, `nuqs` (18).
8. `idb` / `idb-keyval` packages, Jotai-style atoms maintenance, bundle sizes (16, 19).
9. React DevTools Suspense toggle location, SSR `onError` semantics (13).
10. CDN semantics of `stale-while-revalidate` / `stale-if-error` (06, 14).

Newer-major notes

React Router is at v8.5 (the lesson's `react-router` import is correct; `react-router-dom` is gone in v8). TanStack Query 5.104 changed mutation callback signatures (extra trailing `context`, `onMutateResult` third argument). Zod 4 deprecates `.datetime()` / `.safe()`. No newer major exists for RTK (2.x), Zustand (5.x) or XState (5.x) in the sandbox registry.